

SAVID: Secure and Verifiable DNN Inference on Low-End Edge Devices

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous Country

Abstract

Deploying deep neural network (DNN) inference on low-end edge devices raises two key challenges: protecting model confidentiality against a potentially compromised system and enabling verifiable inference without incurring prohibitive overhead. Existing approaches either house (partial) models and inference software within trusted execution environments (TEEs), resulting in high-cost and application-dependent TCB, or execute in untrusted environments, which provide little security.

In this work, we present SAVID, a framework for secure and verifiable DNN inference on constrained edge devices. At its core, SAVID introduces SHANGRI-LA, a new execution abstraction on TrustZone-M TEEs that establishes a third runtime environment with privileges strictly between the Secure and Non-Secure Worlds. SAVID leverages SHANGRI-LA to execute untrusted inference code in the Non-Secure state while leveraging minimal application-agnostic Secure-World support to protect model confidentiality and enable verifiability of inference results.

We realize SAVID on a real-world NUCLEO-L552ZE-Q development board. Our results show that SAVID achieves small TCB, memory footprint, and runtime overhead, making it practical for low-end devices.

CCS Concepts

• **Security and privacy** → **Embedded systems security**; • **Computing methodologies** → *Neural networks*; • **Computer systems organization** → *Embedded systems*.

Keywords

Trusted execution, embedded machine learning, model confidentiality, proof of execution, TrustZone-M

ACM Reference Format:

Anonymous Author(s). 2026. SAVID: Secure and Verifiable DNN Inference on Low-End Edge Devices. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Edge computing has emerged as a dominant paradigm for performing inference of machine learning (ML) models, particularly deep neural networks (DNNs), in latency-sensitive and resource-constrained environments [11]. In this paradigm, DNN models are trained in centralized infrastructures (e.g., the cloud) by service providers, referred to as model providers, and subsequently deployed to edge devices, such as sensor and actuator microcontrollers. These low-cost devices then perform DNN inference locally, enabling real-time decision-making for several modern IoT applications such as remote medical diagnostics, autonomous systems,

and industrial monitoring []. However, moving from centralized to decentralized deployments poses new classes of attacks.

First, training DNN models typically requires substantial computational resources, as well as significant effort in collecting and curating training data. As such, model providers often consider these models to be valuable intellectual property/assets that must be protected [26]. However, when deployed at the edge, DNN models are exposed to stronger adversarial capabilities, where device-controlled adversaries have both physical and software access to the device. This enables such adversaries to: (1) directly extract the model, either at rest (e.g., from on-device flash memory) or at runtime, and (2) mount *model extraction* attacks [23] by reconstructing the deployed model via performing a large number of carefully crafted inference queries. These threats introduce a new challenge of *model confidentiality* for model providers in edge deployments.

Second, edge inference outcomes are often used to drive critical decision-making. For example, in autonomous systems, object detection results are used to identify obstacles (e.g., pedestrians) and trigger braking upon detection. Similarly, in remote medical diagnostics, ECG analysis results may be used to detect cardiac abnormalities and initiate timely intervention []. However, edge devices operate under strict constraints in terms of cost, size, and power, which limits the deployment of sophisticated security mechanisms. As a result, these low-end devices are more vulnerable to malware infestation, leading to a surge in IoT malware targeting edge devices in recent years []. Once compromised, such malware may take control of the entire software stack and produce spoofed inference results to manipulate downstream decisions, potentially causing safety hazards to the physical environment or human lives. To address this second class of attacks, *verifiable inference* is necessary. In traditional centralized settings, this is typically not a major concern, as the model providers have full control over powerful inference devices.

To enable *model confidentiality* with minimal performance overhead, recent work leverages Trusted Execution Environments (TEEs), giving rise to a class of techniques known as *TEE-Shielded DNN Partitioning (TSDP)* [8, 10, 14, 21]. These techniques partition a DNN into two disjoint components: (i) *privacy-sensitive* and (ii) *privacy-insensitive* parts. The privacy-sensitive components are executed within the TEE, while the remaining parts are left unprotected. Despite being high-performant, this line of work suffers from a key limitation: it enlarges the trusted computing base (TCB) by incorporating DNN inference code directly into the TEE, thereby increasing both the software footprint and the attack surface to already-constrained/vulnerable edge devices. Moreover, many of these approaches [] overlook model extraction attacks or address them in ad-hoc fashions, leaving this important issue unaddressed.

On the other hand, prior work on achieving *verifiable inference* either assumes that the entire inference pipeline is executed within TEEs [2, 5, 24] or relies on cryptographic techniques in conjunction with TEEs [22, 27]. The former is often undesirable in edge settings, as DNN models may exceed the limited memory capacity of the TEE. The latter, while more flexible, can introduce significant computational overhead, making it unsuitable for power-constrained devices. More importantly, both approaches still suffer from TCB bloat due to the inclusion of complex and potentially buggy inference code within the TEE. A compromise of such code could escalate into other high-assurance components (e.g., secure key storage, secure software update) that live in the same TEE, as is the case with ARM TrustZone-M – a commonly used TEE for low-end edge devices.

In light of these challenges, we introduce SAVID, a framework that enables *model confidentiality* and *verifiable inference* for DNNs on low-end edge devices. At its core, SAVID leverages TrustZone-M TEE that divides the system into *two* distinct runtime environments: Secure and Normal Worlds. Unlike prior TEE-based approaches, SAVID avoids placing DNN inference code entirely in Secure World.

Instead, we introduce a new abstraction called SHANGRI-LA, which acts as a *third* runtime environment to TrustZone-M. SHANGRI-LA physically resides in Normal World but operates with elevated privileges (enforced by Secure World), allowing it to reuse existing Normal World functionalities and hence avoiding cross-world code duplication. At the same time, SHANGRI-LA ensures the confidentiality and integrity of its code and data, both at rest and during execution, even in the presence of untrusted Normal World. For external inputs, SHANGRI-LA provides a mechanism to securely retrieve data from I/O peripherals that cannot be tampered with by Normal World. Meanwhile, SHANGRI-LA remains less privileged than Secure World, preventing a compromised/buggy SHANGRI-LA from affecting Secure World's components. Finally, SHANGRI-LA introduces only minimal code into Secure World. Critically, this code is application-agnostic, enabling it to be thoroughly tested or formally verified once and then reused across different SHANGRI-LA applications.

Building on the SHANGRI-LA abstraction, we extend it with two key features. First, SHANGRI-LA is augmented with inference authorization, allowing the model provider to control how often inference can be performed; this helps mitigate attacks on *model confidentiality* via *model extraction attacks*. Second, we extend SHANGRI-LA to generate a verifiable proof that accompanies each inference result. This proof allows an external verifier to assess whether: (1) the inference input is authentic; (2) inference is executed correctly on the intended DNN model and authentic input, and (3) the reported inference result corresponds exactly to this execution, thereby achieving *verifiable inference*. Importantly, it is achieved without compromising user privacy, i.e., the proof validation does not rely on the inference input that may contain sensitive user information.

Contributions. Our main contributions are as follows:

- (1) **Third Runtime Environment.** We introduce SHANGRI-LA, a lightweight execution abstraction on top of TrustZone-M. It provides a third runtime environment that is protected from Normal World compromise, while ensuring that any compromise within SHANGRI-LA cannot affect the Secure

World. At the same time, SHANGRI-LA physically lives in Normal World and thus can reuse existing Normal World code, reducing the overall software footprint. We believe SHANGRI-LA is of independent interest beyond secure DNN inference, as it enables third-party developers to deploy arbitrary trusted applications on TrustZone-M without modifying core Secure World components.

- (2) **Secure and Verifiable DNN Inference.** We propose two architectural extensions to SHANGRI-LA to achieve both *model confidentiality* and *verifiable inference* on low-end edge devices. First, we equip SHANGRI-LA with an inference authorization mechanism to mitigate model extraction attacks. Second, we enable SHANGRI-LA to generate verifiable proofs of input retrieval and inference execution, allowing external verifiers to validate results without accessing sensitive input data. Building on these extensions, we design SAVID, a protocol among edge-inference stakeholders that leverages SHANGRI-LA to realize secure and verifiable DNN inference.
- (3) **Practical Assessment.** We implement SAVID on real-world hardware, particularly prototyping and evaluating an SAVID-enabled DEV on a NUCLEO-L552ZE-Q development board []. Our evaluation results demonstrate the practical benefits of SAVID: Compared to prior TEE-based solutions, SAVID eliminates the need to place inference code in the Secure World, reducing the TCB and overall software footprint by $x\%$ and $y\%$, respectively. Moreover, SAVID incurs only negligible runtime overhead, making it well-suited for resource-constrained devices. Finally, we demonstrate real-world applicability through a case study on: **TODO**:

2 Background

We briefly review the background on TrustZone-M, remote attestation and proof-of-execution primitives, as well as DNN partitioning strategies commonly used in TEE-based solutions.

2.1 TrustZone for ARM Cortex-M MCUs

ARM TrustZone-M provides hardware-enforced isolation for Cortex-M microcontrollers by partitioning the system into two domains: the Secure World and the Normal World. Each domain can only access memory regions assigned to its corresponding security state.

Memory regions, including code, data, and peripherals, are classified as Secure, Non-Secure, or Non-Secure Callable (NSC) through the Security Attribution Unit (SAU) and the Implementation-Defined Attribution Unit (IDAU), both configured by Secure-World code at boot time. This ensures that only Secure-World code can access Secure memory, while Non-Secure code is restricted to Non-Secure regions.

To enable controlled invocation, the Secure World exposes APIs that can be called by the Normal World through designated NSC entry points. These entry points are the only locations where control can transfer from the Non-Secure World to the Secure World, preventing arbitrary jumps into Secure-World code. As a result, the Normal World cannot tamper with the execution of Secure-World code except through explicitly defined APIs, and cannot access

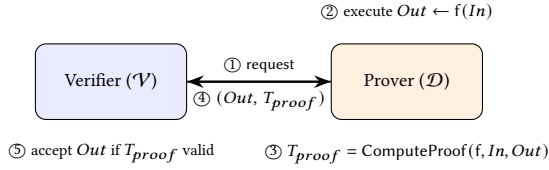


Figure 1: Generic Proof-of-Execution (PoX) workflow.

Secure-World resources protected by the SAU and IDAU, ensuring strong isolation between these two worlds.

In practice, the Secure World contains a *small* trusted computing base (TCB) consisting of security-critical components (e.g., secure boot, key storage, and attestation) to minimize the attack surface. In contrast, the Non-Secure World hosts a significantly larger software stack, including the operating system and user applications.

We also note that, unlike the IDAU, the SAU can be reconfigured at runtime, allowing memory regions to transition dynamically between Secure and Non-Secure states.

2.2 Remote Attestation and Proof of Execution

Remote Attestation. RA allows a remote verifier to establish trust in the software state of a target device [12]. In a TrustZone-M setting, the Secure World acts as the hardware Root of Trust (RoT): it measures the loaded software image and signs the result under a device-bound key sk_D , yielding a report checkable against a known-good reference. RA certifies *what* is loaded, but offers no guarantee over *what the device subsequently computes*.

Proof of Execution. PoX extends RA into the execution domain [16]. The RoT attests not only to code identity but to the fact that a designated function f ran to completion on a given input and produced a given output within the attested environment. As illustrated in Figure 1, the protocol follows a challenge-response exchange between a Verifier V and a Prover D :

- (1) **Challenge.** V issues a request carrying a fresh nonce η and input In .
- (2) **Execution.** D computes $Out \leftarrow f(In)$ inside the Secure World; the RoT enforces that f is neither skipped nor tampered with by Normal-World software.
- (3) **Token generation.** The RoT signs (η, id_f, In, Out) under sk_D , producing a token T_{proof} that binds the nonce, function identity, and input/output pair into a single unforgeable proof.
- (4) **Response.** D returns (Out, T_{proof}) ; V accepts Out as authentic if and only if the signature verifies under the device’s certified public key.

2.3 TEE-Shielded DNN Partitioning

Running a full DNN inside a TrustZone-M TEE is generally infeasible: Secure RAM is measured in tens of kilobytes, whereas even compact models require orders of magnitude more memory. TEE-Shielded DNN Partitioning (TSDP) addresses this by decomposing a model M into two sub-networks [8, 10, 14]: a sensitive component M_{pri} sequestered in the Secure World, and a public component M_{pub} executed in the Normal World under the device’s full memory

and compute budget. Activations cross the security boundary at the partition point; each crossing incurs a context-switch penalty whose cost grows with the number of inter-domain transitions.

Partition objectives. The choice of boundary is dictated by the target security property:

- **IP protection.** TeeSlice [10] and SecureQNN [3] isolate bottleneck layers whose parameters are critical to model utility, raising the cost of surrogate-model reconstruction via extraction or distillation attacks [25].
- **Input privacy.** ShadowNet [21] and SelfTZ [20] shield the initial layers, which are most susceptible to feature-inversion and membership-inference attacks against raw input data.
- **Execution efficiency.** SmartZone [9] minimizes world-switch frequency by colocating layers that exchange large activation tensors, bounding latency to the number of partition crossings rather than model depth.

Regardless of the chosen objective, TSDP requires hosting M_{pri} alongside a full inference engine (e.g., TFLite Micro) inside the Secure World, inflating the TCB beyond the minimal footprint of the hardware RoT. End-to-end latency is similarly constrained: on cycle-limited MCUs, fine-grained partitions that maximize security coverage do so at the cost of frequent pipeline stalls on each domain crossing.

3 Preliminaries

3.1 System Model

We consider an edge inference setting that comprises five stakeholders:

- **Model provider (Pvd):** curates training data, provides the DNN model M , and authorizes access to it.
- **Edge device (Dev):** a small device running with a single-core MCU and no MMU, making it unsuitable for running a monolithic operating system. It is, however, equipped with a low-cost TrustZone-M TEE responsible for safeguarding M .
- **Model consumer (Csm):** obtains authorization from Pvd to run inference on Dev.
- **Device owner (Usr):** owns and has physical access to Dev.
- **Verifier (Vrf):** receives inference results and validates their trustworthiness before downstream use. Vrf can be distinct or any other parties can act as Vrf, e.g., Vrf and Csm may be the same entity, allowing verification right after inference.

As illustrated in Figure 2, secure edge inference consists of four interactions among these stakeholders:

- **I1: Provisioning.** Pvd provisions Dev with required cryptographic material and installs the input-retrieval and inference code, F , along with M on Dev.
- **I2: Authorization.** Csm requests permission from Pvd to use M on Dev. If approved, Pvd issues an authorization token T_{Csm} on M ’s identifier ID_M , encoding Csm’s usage policy on M .
- **I3: Inference.** Csm invokes inference on Dev by presenting T_{Csm} along with the input In . Upon successful validation, Dev executes $F(In)$ with M , producing an inference result Out along with a proof T_{proof} .

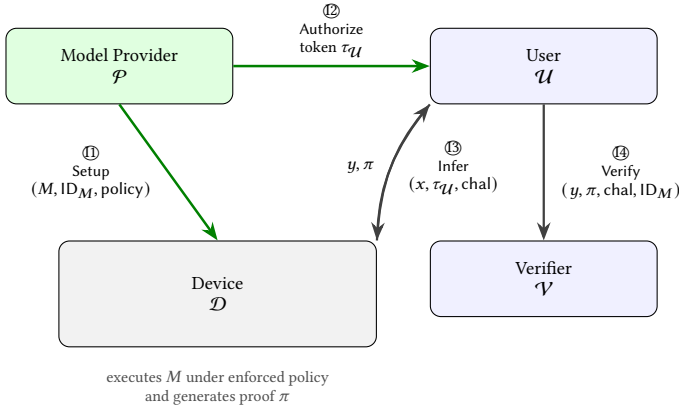


Figure 2: High-level protocol among the model provider $\mathcal{P}$, device $\mathcal{D}$, user $\mathcal{U}$, and verifier $\mathcal{V}$. OAK: make it nicer, the numbers are wrong, the notation is not consistent with the rest of the paper.

- **I4: Proof Verification.** CSM (or a proxy) sends *Out* and T_{proof} to VRF, which validates the authenticity of *Out* with respect to the expected F and T_{proof} , and accepts the result if verification succeeds.

3.2 Threat Model

We consider two types of adversaries $\mathcal{A}_M$ and $\mathcal{A}_I$ that aims to achieve two different goals.

The goal of $\mathcal{A}_M$ is to extract M . We consider $\mathcal{A}_M$ that can compromise all parties, including DEV, CSM, USR, and VRF, except PVD who already possesses M . Furthermore, by compromising DEV that hosts M , $\mathcal{A}_M$ gains full control over its communication channels, enabling message interception, modification, replay, and injection. It also controls all untrusted software on DEV, with the ability to read, write, and execute arbitrary code and data in the Normal World. $\mathcal{A}_M$ may use flaws in I2–I4 in conjunction with compromised DEV to induce unintended leakage beyond what PVD authorizes in I2.

For $\mathcal{A}_I$, its goal is to convince VRF to accept a forged inference result *Out* that was not legitimately produced. We consider $\mathcal{A}_I$ to control DEV, USR, and CSM, with capabilities similar to $\mathcal{A}_M$ and leverage these capabilities to achieve his goal.

Trust Assumptions. We exclude PVD from the threat model: as it already possesses M , it has no incentive to act as $\mathcal{A}_M$, and as it aims to provide correct inference service on DEV, there is no point to behave like $\mathcal{A}_I$.

We assume a trusted public key infrastructure (PKI) that enables PVD, CSM, and VRF to authenticate each other. Due to resource constraints and possible lack of connectivity, we do not assume PKI enforcement on DEV.

DEV is assumed to be equipped with a TrustZone-M TEE, whose Secure World code is trusted and free of vulnerabilities. This assumption motivates our design choice to avoid placing the potentially large function F in the Secure World. Following TrustZone-M’s trust model [18], we assume an underlying secure boot process that guarantees the integrity of Secure World code at boot time

as well as secure persistent storage restricted to Secure World accesses [7].

Since PVD is trusted and it develops F , it is expected to ensure F correctness, i.e., it correctly retrieves input data from properly configured peripherals and performs inference without leaking sensitive information (i.e., M). However, given its size, there is still a chance for F to contain vulnerabilities. In such cases, our security guarantees may no longer hold. Nevertheless, we expect any compromise to not impact Secure World components, which are application-agnostic and more trustworthy.

Out-of-Scope Attacks. Following TrustZone-M’s trust model [18], we do not consider side-channel attacks, invasive hardware attacks or denial-of-service attacks.

3.3 Goals

Given the system and threat models, we aim to design SAVID that realizes three main goals:

G1 (Model Confidentiality): $\mathcal{A}_M$ should not learn any information about M beyond what PVD allows. This goal consists of two sub-goals:

G1-1 (Direct Leakage Protection): Even with compromised DEV, $\mathcal{A}_M$ cannot learn anything about M , except for *Out* and T_{proof} .

G1-2 (Indirect Leakage Protection): Since *Out* may still leak information about M (e.g., via model extraction attacks), $\mathcal{A}_M$ must not be able to invoke F more than what PVD explicitly authorizes.

G2 (Verifiable Inference): An adversary $\mathcal{A}_I$ must not be able to trick VRF into accepting invalid inference results. It consists of two sub-goals:

G2-1 (Authenticity): VRF must be able to detect invalid pairs (*Out*, T_{proof}) submitted by $\mathcal{A}_I$. In particular, this happens when: (1) *Out* is forged without executing F , (2) *Out* is produced by executing F on a different DEV, (3) *Out* is produced by executing F on the right DEV but using a different model than M , or (4) *Out* results from an incomplete execution of F on DEV.

G2-2 (Privacy): To preserve USR’s privacy, VRF must be able to validate (*Out*, T_{proof}) without access to the inference input *In*, which may contain sensitive data.

G3 (Low Overhead): To accommodate DEV’s limited resources, SAVID must incur minimal overhead on DEV. We consider three types of overhead:

G3-1 (TCB): The increase in Secure World TCB size must be small and independent of F , allowing the same TCB to support arbitrary implementations of input retrieval and inference.

G3-2 (Memory): SAVID should minimize memory usage by avoiding duplication of code or model data across Secure and Normal Worlds whenever possible.

G3-3 (Latency): SAVID must not introduce significant runtime overhead during inference.

Table 1: Memory access control of TrustZone-M domains with our introduction of SHANGRI-LA; RO indicates read-only access.

	Normal World		Code	SHANGRI-LA		Secure World	Code	Data
	Code	Data		Priv. Data	Pub. Data			
Normal World	✓	✓	RO	×	RO	×	×	×
SHANGRI-LA	✓	✓	✓	✓	✓	×	×	×
Secure World	✓	✓	✓	✓	✓	✓	✓	✓

4 Design Overview

4.1 Challenges

Simultaneously achieving **G1** and **G3** is challenging. To illustrate this, we focus on a subset of these: **G1-1** and **G3**. Even under this simplification, we show that commonly used designs in the literature – **D1**, **D2**, and **D3** – fail to satisfy both of them. Since SAVID also targets more goals, it faces even more challenges and complexity.

D1: Fully Non-Secure Execution. We start with a strawman approach that executes F entirely in *Normal World*. While this approach naturally satisfies **G3** and is easy to deploy, it provides no security guarantees: $\mathcal{A}_M$ can trivially violate **G1-1**, e.g., by extracting M from DEV 's flash/RAM while $\mathcal{A}_I$ can steal the attestation key in Normal World, resulting in a successful forgery of T_{proof} .

D2: Fully Secure Execution. At the other end of the spectrum, F can be placed and executed entirely in the Secure World. To protect M at rest, PVD can provision it in encrypted form (during **I1: Provisioning**), which is then decrypted and loaded into secure RAM for execution by F post-boot.

This satisfies **G1-1**, not **G3**. In particular, **G3-1 (TCB)** is violated, as the Secure World must include F , which is application-specific (e.g., dependent on input sensors and inference implementations). Additionally, **G3-2 (Memory)** is strained, since DEV must store both an encrypted version of M in flash and a plaintext version in secure RAM. Finally, this design limits the use of emerging ML hardware accelerators (e.g., microNPU's [], Edge TPU's []), which are typically controlled by the Normal World [], thereby hindering potential performance gains.

D3: Split Inference Across Worlds. A middle-ground approach partitions M into private (M_{pri}) and public components (M_{pub}). Here, M_{pri} is provisioned in encrypted form and executed in Secure World, similar to **D2**; M_{pub} is managed and executed in the Normal World, as done in **D1**. This design is commonly used in prior TEE-based work [10] to achieve **G1-1** on potentially untrusted devices.

Since M_{pri} is typically much smaller than M_{pub} , this design reduces secure memory requirements compared to **D2**, partially addressing **G3-2 (Memory)**. However, it does not resolve **G3-1 (TCB)**, as Secure World must still host F to ensure correct usage of M_{pri} . Moreover, **G3-3 (Latency)** can be negatively impacted due to frequent world switches between Secure and Normal Worlds, with overhead depending on the partitioning strategy, e.g., approaches like TEESlice [10], which distribute private components across multiple layers, can incur substantial switching costs. Finally, this design still limits the use of emerging low-cost hardware accelerators for executing M_{pri} .

4.2 High-Level Intuition

From **D1–D3**, we observe that **G3** is achieved only by **D1**. This suggests that extending **D1** with stronger security mechanisms could enable us to achieve both **G1-1** and **G3**.

Guided by this insight, we propose a new abstraction, SHANGRI-LA, which executes F in Normal World, inheriting **D1**'s benefits for achieving **G3**. To enforce **G1-1**, we introduce a small, F -agnostic component in the Secure World that elevates SHANGRI-LA privilege during its execution, placing its privilege between Normal and Secure Worlds.

With this new privilege, Normal World cannot access or tamper with SHANGRI-LA's code and data, ensuring **G1-1**. At the same time, SHANGRI-LA is prevented from interfering with Secure World. As a result, this design enables third-party developers to deploy isolated code without modifying highly trusted Secure World components. Overall, SHANGRI-LA establishes a new, third, runtime environment on TrustZone-M where the privilege relationships among the three environments are summarized in Table 1.

Extensions. To provide complete protection, we extend SHANGRI-LA to support **G1-2** and **G2** without sacrificing **G3**. Specifically, we augment SHANGRI-LA's base design with two features: (1) an authorization mechanism that enables a trusted party to control who can invoke SHANGRI-LA and how often, thereby enforcing **G1-2** (assuming F correctly implements input retrieval and inference over M), and (2) a proof-of-execution mechanism that allows Secure World to generate verifiable evidence that SHANGRI-LA executed correctly and that a given output corresponds to this execution.

5 SHANGRI-LA: Third Runtime Environment on TrustZone-M

Here, we describe SHANGRI-LA, including its lifecycle and Secure-World APIs to orchestrate its lifecycle state.

5.1 Lifecycle of SHANGRI-LA

The lifecycle of SHANGRI-LA (Figure 3a) consists of three states: Non-Exist, Inactive, and Active, with transitions controlled by four Secure World APIs.

On a fresh DEV , SHANGRI-LA is in the Non-Exist state. During provisioning, Provision API is called to install the necessary SHANGRI-LA metadata (e.g., code/data hash and keys), while Authorize configures access control policies such as permitted users and usage limits. These steps prepare SHANGRI-LA for deployment but do not instantiate it, so it remains in the Non-Exist state.

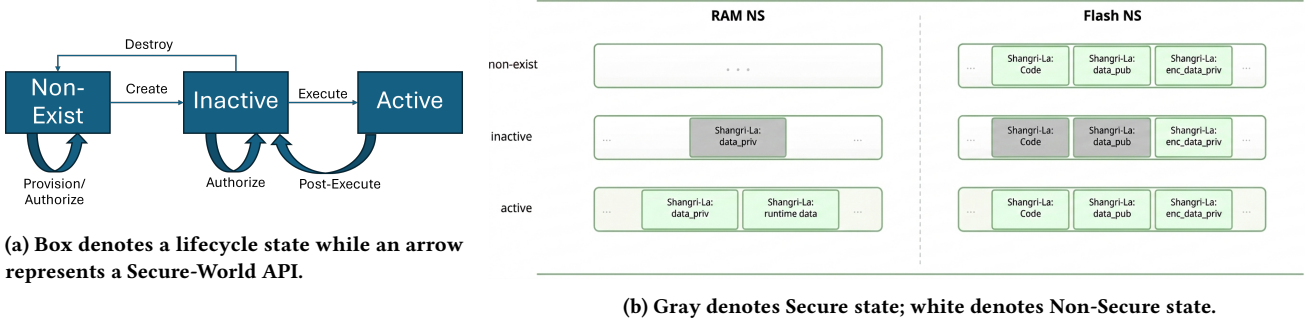


Figure 3: Overview of SHANGRI-LA lifecycle (a) and its corresponding TrustZone-M memory state (b). OAK: make it nicer for Figure 3a too. Figure 3b – make it bigger, it is hard to see, also increase its resolution

To create an executable instance, the Normal World invokes Create. This validates SHANGRI-LA’s code and data with the stored metadata, initializes and assigns it the privilege level defined in Table 1, and transitions it to the Inactive state, where it is ready to run. Invoking Execute transitions SHANGRI-LA to the Active state, during which it executes F. Upon completion, it returns to the Inactive state. This enables repeated executions by alternating between Inactive and Active via Execute. While in the Inactive state, Authorize may be invoked to update SHANGRI-LA authorization policy without requiring re-creation.

When SHANGRI-LA is no longer needed, Destroy can be invoked to erase its sensitive state and release its memory back to the Normal World, returning it to the Non-Exist state.

Next, we describe how aforementioned Secure World APIs are designed/implemented in TrustZone-M.

5.2 Secure-World APIs

To orchestrate the SHANGRI-LA lifecycle, we maintain its context in secure persistent storage and expose a set of Secure-World APIs that allow the Normal World to trigger controlled state transitions without directly accessing or manipulating SHANGRI-LA code and data.

As a basic form, we provide Provision, Create, Execute, and Destroy, which together ensure that SHANGRI-LA adheres to the privilege model in Table 1. We further introduce two optional extensions: (1) an Authorize API that allows the SHANGRI-LA owner to specify which users may access SHANGRI-LA and how many times it can be invoked, and (2) an input flag to Execute that enables generation of a proof of execution, allowing an external verifier to validate the authenticity of the output.

All APIs are summarized in Algorithm 1, while the corresponding TrustZone-M security states of SHANGRI-LA memory across its lifecycle are illustrated in Figure 3b.

5.2.1 Provision (Line 2-5). This API is invoked during a secure provisioning phase. It takes as input the SHANGRI-LA owner’s public key pk_o and a public identifier s_id describing the SHANGRI-LA instance, containing its code F, entry point *entry*, and a data identifier d_id that captures both public data $data_{pub}$ and encrypted private data enc_data_{priv} (if any). If enc_data_{priv} is present, a decryption key k_{dec} is also provided. Optionally, the API accepts a user public

key pk_u and an access limit *limit*, specifying which user is authorized to invoke F within SHANGRI-LA and how many times it may be executed.

Upon invocation, it computes a hash of s_id to serve as a unique identifier for the SHANGRI-LA instance, initializes its Secure-World context with the provided input, sets the instance’s usage counter to 0, and places the instance in the Non-Exist state.

Unlike the other APIs, Provision is only accessible to a trusted entity, e.g., during device manufacturing or via a Secure-World update mechanism that verifies the integrity of the SHANGRI-LA configuration prior to installation.

5.2.2 Authorize (Line 6-12). This API updates the access policy of a provisioned SHANGRI-LA. It takes as input a user public key pk_u , a new limit *limit*, a unique SHANGRI-LA identifier H_{s_id} , and an authorization token T_o over $(pk_u, limit, H_{s_id})$ produced by the SHANGRI-LA owner. Upon invocation, it verifies that a SHANGRI-LA instance identified by H_{s_id} has been provisioned and validates T_o using pk_o . If both checks succeed, it updates the instance’s context with pk_u and *limit* accordingly. This API enables the SHANGRI-LA owner to dynamically change the authorized user and adjust the number of allowed invocations for that user.

5.2.3 Create (Line 13-20). This API initializes a SHANGRI-LA instance from its provisioned configuration, taking s_id as input. Then, it retrieves the memory regions corresponding to the code F, public data $data_{pub}$, and encrypted private data enc_data_{priv} from the instance’s context. It then uses the SAU to mark the code and $data_{pub}$ regions as Secure. If encrypted private data exists, the API allocates a region $data_{priv}$, marks it as Secure, and decrypts enc_data_{priv} using k_{dec} into this region. Finally, it sets the SHANGRI-LA lifecycle state to Inactive, indicating that the instance is ready for execution. As illustrated in Figure 3b, after this call, all SHANGRI-LA code and data are in the Secure state, preventing access or modification by the Normal World.

5.2.4 Execute (Line 21-40). This API performs a single F invocation in SHANGRI-LA. It takes as input a message containing the post-execution usage counter u , F input In , $H(s_id)$, and a signature T_u of this message produced by the user.

Upon invocation, it looks up a SHANGRI-LA instance from H_{s_id} and retrieves its context. It further checks that: (1) the instance is

Algorithm 1: SHANGRI-LA Secure World APIs

```

1 Context:  $CTX : H_{SHANGRI-LA} \rightarrow$ 
    $(k_{dec}, pk_o, pk_u, data\_id, entry, usage, limit, state)$ 

2 Function Provision( $k_{dec}, pk_o, pk_u, s\_id, limit$ )
3    $H_{s\_id} \leftarrow \text{Hash}(s\_id)$ 
4   parse ( $data\_id, entry$ ) from  $s\_id$ 
5    $CTX[H_{s\_id}] \leftarrow$ 
      $(k_{dec}, pk_o, pk_u, data\_id, entry, 0, limit, \text{Non-Exist})$ 

6 Function Authorize( $pk_u, limit, H_{s\_id}, T_o$ )
7   if  $H_{s\_id} \notin CTX$  then
8     abort
9   retrieve  $pk_o$  from  $CTX[H_{s\_id}]$ 
10  if  $limit \leq CTX[H_{s\_id}].limit$  or not Verify( $pk_o, T_o,$ 
      $pk_u || limit || H_{s\_id}$ ) then
11    abort
12   $CTX[H_{s\_id}].\{pk_u, limit\} \leftarrow \{pk_u, limit\}$ 

13 Function Create( $s\_id$ )
14   $H_{s\_id} \leftarrow \text{Hash}(s\_id)$ 
15  if  $H_{s\_id} \notin CTX$  then
16    return failure
17  parse ( $F, data_{pub}, enc\_data_{priv}$ ) from  $s\_id$ 
18  mark ( $F, data_{pub}, data_{priv}$ ) as Secure
19   $data_{priv} \leftarrow \text{AuthDec}(CTX[H_{s\_id}].k_{dec}, enc\_data_{priv})$ 
20   $CTX[H_{s\_id}].state \leftarrow \text{Init}$ 

21 Function Execute( $u, In, H_{s\_id}, proof, T_u$ )
22  if  $H_{s\_id} \notin CTX$  then
23    abort
24   $(pk_o, pk_u, data\_id, entry, usage, limit, state) \leftarrow$ 
      $CTX[H_{s\_id}]$ 
25  if  $state \neq \text{Init}$  or  $usage \geq limit$  or  $u \neq usage + 1$  then
26    abort
27  if not Verify( $pk_u, T_u, u || In || H_{s\_id} || proof$ ) then
28    abort
29  disable interrupts
30   $CTX[H_{s\_id}].state \leftarrow \text{Active}$ 
31  allocate SHANGRI-LA stack
32  mark ( $F, data_{pub}, data_{priv}$ ) as Non-secure
33   $Out \leftarrow \text{execute } entry(In)$ 
34  erase stack
35  mark ( $F, data_{pub}, data_{priv}$ ) as Secure
36   $CTX[H_{s\_id}].\{usage, state\} \leftarrow \{u, \text{Inactive}\}$ 
37  enable interrupts
38  if  $proof$  then
39     $T_{proof} \leftarrow \text{Sign}(sk_{DEV}, F || data\_id || In || Out || pk_o)$ 
40  return ( $Out, T_{proof}$ )

41 Function Destroy( $H_{s\_id}$ )
42  if  $H_{s\_id} \notin CTX$  then
43    abort
44  erase  $data_{priv}$ 
45  mark ( $F, data_{pub}, data_{priv}$ ) as Non-secure
46   $CTX[H_{s\_id}].state \leftarrow \text{Non-Exist}$ 

```

in the Inactive state, (2) the provided counter u is consistent with the current usage (i.e., $u = usage + 1$), (3) the invocation does not exceed $limit$ and (4) authenticity of T_u using pk_u .

If all checks succeed, Secure World prepares execution of F in SHANGRI-LA as follows: it disables interrupts, sets the SHANGRI-LA state to Active, and allocates a dedicated stack by pointing The Main Stack Pointer (MSP) to this location. All other Non-Secure RAM, except the SHANGRI-LA stack and its data regions ($data_{pub}$ and $data_{priv}$), is marked as Secure.

Next, Secure World marks F , $data_{pub}$ and $data_{priv}$ as Non-Secure (see Figure 3b) and transfers control to Normal World at the entry point $entry$ to execute F with input In . Once F execution completes, control returns to Secure World. It then zeroes the SHANGRI-LA stack, restores SHANGRI-LA code and data to Secure state, increments the usage counter, and sets the lifecycle state back to Inactive before re-enabling interrupts.

If proof of execution is required, Secure World generates PoX by signing ($F, data_id, In, Out$) using the device-bound attestation key sk_{DEV} . Finally, it returns (PoX, Out) to the caller.

5.2.5 Destroy (Line 41-46). This API tears down a SHANGRI-LA instance H_{s_id} . Upon invocation, Secure World erases all sensitive data in $data_{priv}$. It then marks F , $data_{pub}$ and $data_{priv}$ regions as Non-Secure, releasing the memory back to the Normal World. Finally, it sets the SHANGRI-LA lifecycle state to Non-Exist.

5.3 Security Analysis

We argue that aforementioned SHANGRI-LA lifecycles and Secure-World APIs achieve the following security properties:

P1: Privilege Separation. We show that the four core APIs – Provision, Create, Execute (without proof-of-execution), and Destroy – are sufficient to enforce the privilege model in Table 1.

During Execute, F runs atomically with interrupts disabled, and control is not returned to Normal World until both F and Execute complete. This prevents Normal World from interrupting execution to access $data_{pub}$ or $data_{priv}$ (which are in Non-Secure state) while SHANGRI-LA is in the Active state.

A compromised Normal World may attempt to access SHANGRI-LA code and data outside the Active state. When SHANGRI-LA is in the Inactive state, such access is prevented by the SAU, as all SHANGRI-LA memory regions are marked Secure (Figure 3b). The remaining opportunity is when SHANGRI-LA is in the Non-Exist state. In this state, F and $data_{pub}$ reside in Non-Secure flash and may be read or modified. However, any tampering is detected during Create, as the recomputed H_{s_id} (over F and $data_{pub}$) will not match the provisioned value, preventing SHANGRI-LA from being created. For $data_{priv}$, confidentiality is preserved since it exists only in encrypted form in the Non-Exist state.

A subtle case arises if the device resets while SHANGRI-LA is in the Active or Inactive state, where $data_{priv}$ or its derived runtime state may persist in Non-Secure RAM. To address this, we propose either: (i) extending Secure-World initialization to clear all Non-Secure RAM at boot, or (ii) having Execute record the stack region in its context and extending Secure-World initialization to detect incomplete executions and securely erase the corresponding memory.

We note that SHANGRI-LA always lives/executes in the Non-Secure state and therefore cannot access or interfere with higher-assurance Secure-World components. At the same time, executing in the Non-Secure state allows SHANGRI-LA to access and reuse existing Non-Secure code when needed. As a result, these satisfy the privilege model in Table 1.

P2: Authorization of SHANGRI-LA Invocation. We show that, with the addition of the Authorize API, only authorized users can invoke F in SHANGRI-LA, and they cannot exceed the usage limit specified by the owner.

An unauthorized user cannot execute F because their public key is not registered in the SHANGRI-LA context. The Execute API requires a user message signed by the authorized user and verifies it against the authorized user's public key pk_u . Without access to the corresponding private key, an adversary cannot produce a valid signature and thus cannot run a SHANGRI-LA instance.

Furthermore, Authorize updates the authorized user and usage limit only upon receiving a valid owner-signed message (verified using pk_o). The usage counter is restricted to Secure World and checked during Execute to ensure that each invocation is fresh and does not exceed the specified limit. This prevents both overuse and replay attacks.

P3: PoX Unforgeability. Finally, when the proof-of-execution feature is enabled ($proof = 1$ in Execute), we show that a valid PoX can only be produced if F is executed correctly within SHANGRI-LA. We adopt the notion of correct execution from prior work [16], where F must execute completely from its entry point to its exit point without interruption.

PoX is generated inside Secure World via the Execute API using sk_{DEV} and covers $(F, data_id, In, Out)$. Since sk_{DEV} is not accessible to the Normal World, an adversary cannot forge a valid PoX. Consequently, a valid PoX can only be obtained by successfully invoking Execute, which runs F with input In inside SHANGRI-LA to produce the corresponding output Out . Any attempt to modify In or Out results in an invalid proof, as both are cryptographically bound in PoX. Moreover, interrupting F execution to induce an incomplete execution is not possible due to interrupts being disabled during F execution. A reset during Execute may cause F to terminate prematurely; however, in this case, no PoX is produced, since it is generated only upon successful completion of Execute. In all cases, an adversary cannot forge PoX without having to execute F inside SHANGRI-LA with the correct In .

6 SAVID: Edge Inference Stakeholder Interactions

Building on the SHANGRI-LA abstraction and its Secure-World APIs (Section 5), we present SAVID, a protocol that realizes secure edge inference interactions I1–I4 defined in Section 3. The protocol is summarized in Algorithm 2.

As mentioned in Section 4, we assume a trusted PKI in which each stakeholder $x \in \{PVD, CSM, DEV, VRF\}$ possesses a key pair (sk_x, pk_x) . All stakeholders know each other's public keys, except that DEV does not maintain or rely on external public keys beyond those provisioned through its Secure-World context.

Algorithm 2: SAVID Protocol

```

1 I1: Provisioning (Pvd → Dev)
   // Executed by Pvd
2    $(M_{pub}, M_{pri}) \leftarrow \text{TDSP-Partition}(M)$ 
3    $s\_id \leftarrow (F, entry, M_{ID}, M_{pub}, \text{AuthEnc}(k_{dec}, M_{pri}))$ 
   // Executed on Dev by a trusted entity
4   Install F,  $data_{pub}$ ,  $enc\_data_{priv}$  in Normal World
5   Provision( $k_{dec}, pk_{PVD}, pk_{CSM}, s\_id, limit$ )

6 I2: Authorization (Pvd ↔ Csm → Dev)
   // Executed by Pvd, after verifying Csm's request
7    $T_{PVD} \leftarrow \text{Sign}(sk_{PVD}, pk_{CSM} \parallel limit \parallel H_{s\_id})$ 
   // Executed on Dev
8   Authorize( $pk_{CSM}, limit, H_{s\_id}, T_{PVD}$ )

9 I3: Inference (Csm ↔ Dev)
   // Executed by Csm
10   $T_{CSM} \leftarrow \text{Sign}(sk_{CSM}, u \parallel In \parallel H_{s\_id} \parallel proof)$ 
   // Executed on Dev
11   $(Out, T_{proof}) \leftarrow \text{Execute}(u, In, H_{s\_id}, proof, T_u)$ 

12 I4: Proof Verification (Csm → Vrf)
13  if not Verify( $pk_{DEV}, T_{proof}, F \parallel M_{ID} \parallel In \parallel Out \parallel pk_{PVD}$ )
14    or not IsValid( $F, M_{ID}, In, pk_{PVD}$ ) then
15    reject Out
16  else
17    accept Out

```

6.1 I1–I4 in SAVID

I1: Provisioning. PVD prepares the software F, whose entry point *entry*, which implements both input retrieval from DEV's peripherals and DNN inference. Given a DNN model M , PVD assigns its ID as M_{ID} and then applies a TDSP partitioning strategy to split it into private layers M_{pri} and public layers M_{pub} . It then generates a symmetric key k_{dec} and uses it to encrypt M_{pri} .

Using s_id , PVD installs F, M_{pub} , and encrypted M_{pri} in the Normal World of DEV, while provisioning the corresponding SHANGRI-LA metadata to the Secure World via the Provision API. Besides s_id and k_{dec} , this metadata also includes PVD's public key pk_{PVD} , and optionally the authorized user's public key pk_{CSM} together with the usage limit *limit*.

I2: Authorization. CSM requests permission from PVD to invoke inference on DEV. If approved, PVD issues a signed authorization token T_{CSM} that grants the user with public key pk_{CSM} access to the SHANGRI-LA instance identified by H_{s_id} with a usage limit *limit*.

CSM forwards T_{CSM} to DEV, which invokes the Authorize API to update the access policy of the corresponding SHANGRI-LA instance. Upon successful verification of T_{CSM} , the Secure World registers pk_{CSM} and *limit* for H_{s_id} .

I3: Inference. To perform inference, CSM first invokes the Create API to instantiate the SHANGRI-LA instance identified by H_{s_id} . It then constructs a request containing the next usage counter u , the input In , the identifier H_{s_id} , and a flag $proof = 1$ indicating that a

proof of execution is required. This request is signed using sk_{CSM} to produce an inference token T_{proof} .

CSM submits the request and T_{proof} to the Execute API. Upon successful verification, Execute runs the corresponding function F within SHANGRI-LA, which retrieves input data (e.g., from a sensor) and performs inference using M , producing an output Out . If $proof = 1$, the API additionally returns a proof of execution T_{proof} .

I4: Proof Verification. CSM forwards (Out, T_{proof}) to V_{RF} . V_{RF} checks that T_{proof} is a valid token under pk_{DEV} over the expected pk_{PVD} , F , M_{ID} , In , and Out .

6.2 Security Analysis

We show that SAVID satisfies **G1** and **G2**. Our arguments build on the security properties of SHANGRI-LA established in Section 5.3.

G1-1 (Direct Leakage Protection). The DNN model M is partitioned into M_{pub} and M_{pri} , where M_{pub} is by design made public by PVD. Thus, **G1-1** concerns only the protection of M_{pri} .

In SAVID, M_{pri} is treated as private data within SHANGRI-LA and therefore inherits **P1: Privilege Separation** that guarantees the access control guarantees in Table 1. In particular, untrusted Normal World on DEV cannot access M_{pri} in plaintext. Outside DEV, the only point at which M_{pri} is handled in plaintext is during **I1: Provisioning** by PVD. As $\mathcal{A}_M$ excludes PVD, $\mathcal{A}_M$ cannot directly observe M_{pri} , satisfying **G1-1**.

G1-2 (Indirect Leakage Protection). Indirect leakage through overused or unauthorized inference queries is mitigated through **I2: Authorization** in SAVID. Therein, CSM must obtain a valid T_{CSM} from PVD, which will be enforced on DEV by the Authorize API before any inference can be executed through Execute. As a result, SAVID inherits **P2: Authorization of SHANGRI-LA Invocation**, preventing unauthorized $\mathcal{A}_M$ from performing inference on M more than PVD's specified usage limit.

G2-1 (Authenticity). In SAVID, **I3: Inference** requires invoking the Execute API, thereby inheriting **P3: PoX Unforgeability** of SHANGRI-LA. In particular, any attempt by $\mathcal{A}_I$ to forge an inference result Out or its corresponding proof T_{proof} will be detected by V_{RF} .

Moreover, since F encapsulates both input retrieval and inference, a valid T_{proof} guarantees that the inference input is obtained without being tampered with and that F executes completely without interruption. As T_{proof} also covers authentic M_{ID} and pk_{PVD} (protected on DEV through Secure World's secure storage), V_{RF} can determine whether the inference is performed using the right DNN model M . In sum, SAVID assures that Out is the authentic result of executing F on input In using the model identified by M_{ID} , as provisioned by the legitimate model provider, thereby fulfilling **G2-1**.

G2-2 (Privacy). In **I4: Proof Verification**, verification relies on public information, including F , In , M_{ID} , pk_{PVD} , Out and T_{proof} . Importantly, the raw inference input is neither revealed nor required during verification, as it is acquired and processed internally within SHANGRI-LA, which is also not exposed to DEV's Normal World. Hence, SAVID preserves the privacy of inference USR inputs, satisfying **G2-2**.

7 Evaluation

OAK: It should show that G3 is satisfied.

This section evaluates the practicality of SAVID on a real TrustZone-M microcontroller, with a focus on performance overhead, memory footprint, and security enforcement effectiveness.

7.1 Implementation & Experimental Setup

We implement SAVID on the STM32L552ZE-Q (ARM Cortex-M33), which provides hardware-enforced Secure/Non-Secure isolation through SAU/IDAU attribution. The Secure World runs TF-M and exposes PSA-based secure services; the Non-Secure World hosts the application runtime, peripheral drivers (e.g., camera), and the DNN inference engine.

Connecting F to decrypted private weights. In our prototype, Create decrypts M_{pri} into a dedicated Secure-marked RAM buffer (38.6 KB for our CIFAR-10 model) and patches the late-layer weight pointers of F to point into that buffer before finalizing the SHANGRI-LA instance. F itself requires no modification: it treats M_{pri} as ordinary weight data, unaware that the buffer is transiently opened to Non-Secure access only during Execute.

SAU region usage. SAVID manages five SAU regions at runtime: Region 0 (NS flash baseline), Region 1 (NS RAM baseline), Region 5 (NS flash beyond the split point), and Regions 6–7 (NS RAM windows bracketing the SHANGRI-LA buffer). Regions 2–4 are reserved for TF-M (NSC vector table, peripherals) and are never modified by SAVID. These five regions protect three artifact classes: private weights in SHANGRI-LA RAM, the model read-only flash window, and the inference code window.

Model size and partitioning. Our evaluation targets models of approximately 100 KB, with a confidential component of ≈ 40 KB (39 552 B encrypted blob). The split point is chosen following standard TSDP criteria [10, 14]; in principle, any TSDP strategy can be applied, as SAVID only requires that the private layers be supplied in encrypted form during Provision.

7.2 Performance and Resource Overhead

We evaluate the overhead introduced by confidentiality and integrity mechanisms by comparing SAVID against three baseline approaches:

- **Naïve baseline (NS-only):** inference executed fully in the Non-Secure world without any protection.
- **Fully Secure baseline (S-only):** inference executed entirely inside the Secure World, providing strong isolation but incurring high memory pressure and context-management overhead.
- **SAVID:** supervised enclave inference with Secure World enforcement, authorization checks, controlled inference policies, and proof generation.

We measure inference latency, cycle counts, and memory footprint in order to quantify the additional cost required to guarantee confidentiality and verifiability. In particular, the fully Secure baseline highlights the impracticality of placing the full inference pipeline inside the Secure World on constrained microcontrollers,

Table 2: Benchmark results comparing baseline inference against SAVID.

Model	NS Latency	S Latency	SAVID Latency	Overhead
CIFAR-10	46.62 ms	46.64 ms	46.73 ms	0.11 ms (0.24

while SAVID aims to achieve comparable security guarantees with significantly lower overhead.

Benchmarking. We benchmark multiple models and configurations to evaluate scalability across different inference workloads. We report results in Table 2, including comparisons between the naïve baseline, the fully Secure baseline, and SAVID.

7.3 Security Validation Tests

Beyond performance, we evaluate whether SAVID correctly enforces the intended security guarantees under adversarial conditions. We implemented a set of negative tests to ensure that unauthorized behavior is detected and rejected.

Memory Isolation Tests. We validate that the Non-Secure world cannot access enclave-protected memory when the enclave is closed. In particular, we test:

- Attempting to read enclave SRAM regions from Non-Secure code.
- Attempting to read protected Flash regions mapped to enclave code/data.

All unauthorized accesses are correctly blocked, confirming that enclave memory protection is enforced as intended.

Protocol Robustness Tests. We also validate that the protocol resists common integrity attacks:

- **Replay attacks:** replaying the same inference request message must fail due to freshness enforcement.
- **Budget enforcement:** attempting inference when the inference counter reaches the maximum authorized limit must fail.
- **Metadata tampering:** providing an invalid enclave information hash (e.g., incorrect EnclaveInfo binding) must be rejected.

All tests successfully triggered rejection, confirming that SAVID enforces authorization, freshness, and model binding.

7.4 Enclave Management Constraints

Finally, we evaluate the practicality of enclave lifecycle management. In the current implementation, SAVID supports only one active enclave instance at a time. To deploy a new model, the system must destroy the existing enclave and create a new one, ensuring that stale enclave state is erased and that policies are re-provisioned securely.

This design simplifies enforcement and reduces the Trusted Computing Base, at the cost of limiting concurrent model execution.

7.5 Summary

Overall, our evaluation demonstrates that SAVID is feasible on constrained TrustZone-M devices. It provides confidentiality and

integrity guarantees through supervised enclave execution and proof generation, while remaining compatible with low-end hardware constraints. Our benchmarks and security validation tests show that SAVID introduces manageable overhead and correctly rejects unauthorized access and malicious protocol deviations.

8 Related Work

Table 3 summarizes how existing systems address **G1-G3**.

TEE-Shielded DNN Partitioning. As discussed in Section 3, TEE-shielded DNN partitioning (TSDP) approaches [8, 10, 14, 21] protect sensitive model components by executing them inside a TEE, but this typically increases Secure-World code size and incurs world-switch overhead. ASGARD [15] extends this direction with virtualization-based isolation, but targets more capable platforms and still requires placing inference logic inside the trusted computing base. Moreover, these systems do not explicitly enforce a query budget (**G1-2**) and do not provide privacy-preserving verifiable inference as required by **G2**.

Verifiable Inference. Prior work has explored verifiable inference through both TEE-based and cryptographic approaches. Systems such as Graviton [24], Laminator [5], and PAL [2] rely on executing the pipeline inside a trusted environment, achieving authenticity and verification goals but increasing trusted code and memory requirements, which conflicts with **G3-1** and **G3-2** on MCU-class platforms. Slalom [22] and VeriSplit [27] reduce TCB dependence by incorporating cryptographic integrity checks, but introduce additional per-inference overhead that is difficult to accommodate on resource-constrained devices (**G3-3**). In contrast, SAVID achieves **G2** through a lightweight proof-of-execution mechanism that avoids cryptographic verification overhead on the device during inference.

Proof of Execution. APEX [16] provides proof-of-execution guarantees on embedded devices under full software compromise. SAVID builds on this line of work by binding proofs to model identity and inference context, and by integrating authorization and usage-limit enforcement to support **G1-2**.

Enclave-based TEEs. Penglai [6] provides scalable enclave memory protection on RISC-V platforms using MMU-based mechanisms. In contrast, SAVID targets Cortex-M devices where such MMU features are unavailable, and instead relies on TrustZone-M primitives (SAU/IDAU) to enforce isolation while keeping SHANGRI-LA in the Non-Secure state to minimize the Secure-World TCB.

OAK: TODO: cite <https://ieeexplore.ieee.org/document/10621088> / <https://www.ndss-symposium.org/wp-content/uploads/2025-449-paper.pdf>

OAK: TODO: Penglai is also related to our work: <https://www.usenix.org/system/files/osdi21-feng.pdf> **kiri: done**

We discuss prior work on trusted execution for edge ML, model protection, remote attestation, and verifiable inference systems.

OAK: have a table showing how each existing work fulfills which goals in **G1-3** **kiri: IA generated, we need second check**

Table 3: Comparison against SAVID’s goals. G1-1: direct leakage protection; G1-2: query budget enforcement; G2-1: verifiable authenticity; G2-2: input privacy during verification; G3-1: no inference code in Secure World; G3-2: no code/data duplication; G3-3: low inference latency. ✓ satisfied; × not satisfied.

System	G1-1	G1-2	G2-1	G2-2	G3-1	G3-2	G3-3
Darknetz [14]	✓	×	×	×	×	×	×
Hou et al. [8]	✓	×	×	×	×	×	×
ShadowNet [21]	✓	×	×	×	×	×	×
TeeSlice [10]	✓	×	×	×	×	×	×
ASGARD [15]	✓	×	×	×	×	×	×
Graviton [24]	✓	×	✓	✓	×	×	×
Laminator [5]	✓	×	✓	✓	×	×	×
PAL [2]	✓	×	✓	✓	×	×	×
Slalom [22]	×	×	✓	✓	×	×	×
VeriSplit [27]	✓	×	✓	✓	×	×	×
APEX [16]	×	×	✓	×	✓	✓	✓
Penglai [6]	✓	×	×	×	✓	✓	✓
SAVID (ours)	✓	✓	✓	✓	✓	✓	✓

9 Discussion

TODO: symmetric-key version for DEV TODO: enable interrupts during F execution in SHANGRI-LA TODO: maybe say why we need a new runtime environment? kiri: done

Symmetric-Key Variant for DEV

SAVID currently relies on ECDH to establish fresh session keys between the Verifier and the Secure World for protecting update-related messages. While this provides forward secrecy and limits the impact of long-term key compromise, it incurs non-trivial latency on resource-constrained devices. A symmetric-key variant could instead rely on pre-shared keys provisioned at deployment time, significantly reducing handshake overhead. However, such a design complicates key rotation and revocation, and makes key leakage more damaging since the secret must persist on the device. We therefore favor the current ECDH-based approach for stronger security guarantees, while exploring hybrid designs as future work.

Interrupt Handling During $\mathcal{F}$ Execution in SHANGRI-LA

Our current prototype disables interrupts during $\mathcal{F}$ execution in SHANGRI-LA to avoid race conditions and preserve the integrity of the secure execution flow. However, this design reduces responsiveness and contributes to runtime overhead. Enabling interrupts would allow better coexistence with real-time workloads but requires careful protection of shared state (e.g., SAU configuration and monotonic counters) to prevent TOCTOU attacks or malicious interference through interrupt handlers. A possible direction is to split inference into preemption-safe segments and allow interrupt handling only at well-defined synchronization points.

Why a New Runtime Environment?

Existing TrustZone-M deployments typically enforce a binary separation between the Secure and Non-Secure Worlds. Executing the full inference stack in the Secure World yields strong protection but leads to a large and application-dependent TCB, while

executing it in the Non-Secure World offers insufficient guarantees. Shangri-La introduces an intermediate runtime environment that enables minimal, application-agnostic Secure-World support to enforce confidentiality and verifiability, while keeping the inference logic untrusted. This design achieves a small TCB without sacrificing strong security properties.

10 Conclusion

Providing strong security guarantees for machine learning inference on low-end edge microcontrollers remains a challenging problem due to strict resource constraints and the exposure of devices to adversarial environments. In this work, we presented SAVID, a system-level architecture that enables confidential and verifiable inference on TrustZone-M platforms.

Our design introduces a protected execution environment (enclave) deployed in the Non-Secure domain but strictly supervised by the Secure World. This approach preserves a small Trusted Computing Base while preventing direct leakage of sensitive model assets. To further mitigate indirect leakage through repeated inference queries, SAVID enforces controlled inference policies managed by the model provider.

Finally, SAVID integrates a lightweight Proof-of-Execution mechanism that enables external entities to verify that inference results were produced by the intended device and model under the enforced security policies. Together, these mechanisms provide a practical balance between confidentiality, integrity, and deployability for embedded AI deployments.

Overall, SAVID demonstrates that strong model protection and verifiable inference can be achieved on constrained microcontroller-class devices through careful system partitioning and secure supervision. OAK: please do not use AI to generate bibtex references. The conference has rejected many papers based on the reference hallucination. Use Google Scholar → "Cite" → BibTeX to get a BibTeX entry for a given paper and copy/paste it to .bib

References

- [1] Sebastian P Bayerl, Tommaso Frassetto, Patrick Jauernig, Korbinian Riedhammer, Ahmad-Reza Sadeghi, Thomas Schneider, Emmanuel Stapf, and Christian Weinert. 2020. Offline model guard: Secure and private ml on mobile devices. In 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE).
- [2] Prach Chantasantitam, Adam Ilyas Caulfield, Vasisht Duddu, Lachlan J Gunn, and N Asokan. 2026. PAL* M: Property Attestation for Large Generative Models. *arXiv preprint arXiv:2601.16199* (2026).
- [3] Miguel Costa, Tiago Gomes, and Sandro Pinto. n.d.. SecureQNN: Shielding the Intellectual Property of QNNs in TinyML Systems. Venue and DOI to be completed.
- [4] Piet De Vaere, Felix Stöger, Adrian Perrig, and Gene Tsudik. 2024. The SA4P framework: Sensing and actuation as a privilege. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 873–885.
- [5] Vasisht Duddu, Lachlan J Gunn, and N Asokan. 2024. Laminator: Verifiable ML property cards using hardware-assisted attestations. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*. 317–328.
- [6] Erhu Feng, Xu Lu, Dong Du, Bicheng Yang, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2021. Scalable memory protection in the {PENGGLAI} enclave. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 275–294.
- [7] Javier Gonzalez and Philippe Bonnet. 2014. Tee-based trusted storage. (2014).
- [8] Jiahui Hou, Huiqi Liu, Yunxin Liu, Yu Wang, Peng-Jun Wan, and Xiang-Yang Li. 2021. Model protection: Real-time privacy-preserving inference service for model privacy at the edge. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2021), 4270–4284.
- [9] Zhaolong Jian, Xu Liu, Qiankun Dong, Longkai Cheng, Xueshuo Xie, and Tao Li. n.d.. SmartZone: Runtime Support for Secure and Efficient On-Device Inference on ARM TrustZone. Venue and DOI to be completed.
- [10] Ding Li, Ziqi Zhang, Mengyu Yao, Yifeng Cai, Yao Guo, and Xiangqun Chen. 2025. Teeslice: Protecting sensitive neural network models in trusted execution environments when attackers have pre-trained models. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–49.
- [11] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2019. Edge AI: On-demand accelerating deep neural network inference via edge computing. *IEEE transactions on wireless communications* 19, 1 (2019), 447–457.
- [12] James Ménétrey, Christian Göttel, Anum Khurshid, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, and Shahid Raza. 2022. Attestation mechanisms for trusted execution environments demystified. In *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, 95–113.
- [13] Josh Millar, Yushan Huang, Sarab Sethi, Hamed Haddadi, and Anil Madhavapeddy. n.d.. Benchmarking Ultra-Low-Power μ NPU. Venue and DOI to be completed.
- [14] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. Darknetz: towards model privacy at the edge using trusted execution environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*. 161–174.
- [15] Myungsuk Moon, Minhee Kim, Joonkyo Jung, and Dokyung Song. 2025. ASGARD: Protecting On-Device Deep Neural Networks with Virtualization-Based Trusted Execution Environments.. In *NDSS*.
- [16] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, and Gene Tsudik. 2020. {APEX}: A verified architecture for proofs of execution on remote devices under full software compromise. In *29th USENIX Security Symposium (USENIX Security 20)*. 771–788.
- [17] Ivan De Oliveira Nunes, Seoyeon Hwang, Sashidhar Jakkamsetti, and Gene Tsudik. 2022. Privacy-from-birth: Protecting sensed data from malicious sensors with VERSA. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2413–2429.
- [18] Sandro Pinto and Nuno Santos. 2019. Demystifying arm trustzone: A comprehensive survey. *ACM computing surveys (CSUR)* 51, 6 (2019), 1–36.
- [19] Norrathep Rattanavipanon and Ivan De Oliveira Nunes. n.d.. SLAPP: Poisoning Prevention in Federated Learning and Differential Privacy via Stateful Proofs of Execution. Venue and DOI to be completed.
- [20] SeLTZ Authors. n.d.. SeLTZ: Fine-Grained Data Protection for Edge Neural Networks Using Selective TrustZone Execution. Authors, venue, and DOI to be completed.
- [21] Zhichuang Sun, Ruimin Sun, Changming Liu, Amrita Roy Chowdhury, Long Lu, and Somesh Jha. 2023. Shadownet: A secure and efficient on-device model inference system for convolutional neural networks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1596–1612.
- [22] Florian Tramer and Dan Boneh. 2018. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287* (2018).
- [23] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security 16)*. 601–618.
- [24] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on {GPUs}. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 681–696.
- [25] Jinwen Wang, Yuhao Wu, Han Liu, Bo Yuan, Roger Chamberlain, and Ning Zhang. n.d.. IP Protection in TinyML. Venue and DOI to be completed.
- [26] Mingfu Xue, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2021. Intellectual property protection for deep learning models: Taxonomy, methods, attacks, and evaluations. *IEEE Transactions on Artificial Intelligence* 3, 6 (2021), 908–923.
- [27] Han Zhang, Zifan Wang, Mihir Dhamankar, Matt Fredrikson, and Yuvraj Agarwal. 2024. Verisplit: Secure and practical offloading of machine learning inferences across iot devices. *arXiv preprint arXiv:2406.00586* (2024).